ARCHITECTURAL SPECIFICATION

DOC ID: WC-ARCH-2026-VOL1

WildCard 게임 아키텍처 분석 보고서 (Vol. 1)

시스템 개요, 전용 서버 권한 토폴로지 및 메타 서브시스템 아키텍처

프로젝트:	WildCard (Unreal Engine 5.7 Dedicated Server)	**작성 일시:**	2026-10-19
타겟 플랫폼:	Windows x64 / Linux Dedicated Server	**주요 모듈:**	WildCard, WildCardEditor
문서 범위:	시스템 전체 개요, 기술 스택, C++ 모듈 의존성, 전용 서버 네트워크 권한 모델, 인증/덱/매치메이킹 서브시스템		

1. 프로젝트 개요 및 아키텍처 설계 사상

1.1. 게임 디자인 및 핵심 비전

WildCard는 언리얼 엔진 5.7 기반으로 개발된 **턴제 전술 카드 배틀(Turn-Based Tactical Card Battler)** 게임입니다. 기존 TCG/CCG 장르의 카드 덱 빌딩 및 자원 관리 요소와 육각형 그리드(Hexagonal Grid) 타일 위에서의 공간 전술 및 유닛 전투 요소를 결합한 하이브리드 게임 시스템을 특징으로 합니다.

특히 게임플레이의 긴장감과 심리전을 극대화하기 위해, 턴마다 비밀리에 행동을 결정하는 **동시 결정(Simultaneous Private Commitment)** 메커니즘과, 공개된 행동에 대해 즉각적인 카운터 및 마법을 발동할 수 있는 **유희왕 스타일의 역순 체인(LIFO Chain Stack)** 공방 시스템을 채택하고 있습니다.

■ WildCard 핵심 게임플레이 루프 3대 축

1. 자원 & 카드 덱 시스템

포스(Force) 및 마나(Mana)의 이중 자원 관리, 26~30장 규격화된 카드 덱, 성유물(Artifact) 내구도 기반 지속 효과.

2. 헥사곤 전장 전술

축 좌표계(Q, R) 기반 육각 타일맵, 3D 카드 빌보드 유닛, 영웅 유닛(Hero Unit)의 고유 능력 및 직업별 전술 스킬.

3. 이중 페이즈 체인 공방

비공개 행동 동시 제출 (Action_Private) 후 전장 미리보기 및 체인 스택(Action_Public)을 거쳐 최종 일괄 동시 적용.

1.2. 모듈 구조 및 UBT 빌드 구성

WildCard 프로젝트는 런타임 게임플레이 로직을 담당하는 주 모듈인 WildCard와, 레벨 및 타일맵 제작을 지원하는 커스텀 슬레이트 툴인 WildCardEditor 모듈로 명확하게 분리되어 있습니다.

Source/WildCard/WildCard.Build.cs C# / UBT

```csharp
// Copyright Epic Games, Inc. All Rights Reserved.
using UnrealBuildTool;

public class WildCard : ModuleRules
{
    public WildCard(ReadOnlyTargetRules Target) : base(Target)
    {
        PCHUsage = PCHUsageMode.UseExplicitOrSharedPCHs;

        PublicDependencyModuleNames.AddRange(new string[]
        {
            "Core", "CoreUObject", "Engine", "InputCore", "EnhancedInput", "UMG",
            "ModelViewViewModel", "GameplayTags", "Json", "JsonUtilities"
        });

        PrivateDependencyModuleNames.AddRange(new string[] { "Slate", "SlateCore" });

        PublicIncludePaths.AddRange(new string[]
        {
            "WildCard", "WildCard/InGame/Game", "WildCard/InGame/Interface",
            "WildCard/InGame/Battle", "WildCard/InGame/Data", "WildCard/InGame/Unit",
            "WildCard/InGame/Card", "WildCard/InGame/UI", "WildCard/InGame/UI/Widget",
            "WildCard/InGame/UI/ViewModel", "WildCard/InGame/Map", "WildCard/InGame/Artifact",
            "WildCard/InGame/Effect", "WildCard/Auth", "WildCard/Deck", "WildCard/Matchmaking"
        });

        if (Target.bBuildEditor)
        {
            PrivateDependencyModuleNames.AddRange(new string[]
            {
                "UnrealEd", "LevelEditor", "Blutility", "UMGEditor",
                "ModelViewViewModelBlueprint", "ModelViewViewModelEditor"
            });
        }
    }
}
```

의존성 모듈	도입 목적 및 아키텍처적 역할
ModelViewViewModel	언리얼 엔진 5.7의 반응형 MVVM 프레임워크를 적용하여 UI 렌더링(UMG)과 게임 상태 데이터 모델을 완전 격리.
GameplayTags	하드코딩된 열거형 대신 계층형 태그를 통해 카드 타입, 키워드, 비주얼 큐 연출(VisualCueTag)을 런타임 확장 가능하게 설계.
EnhancedInput	탑다운 전술 시점 조작(카메라 패닝, 회전, 줌) 및 3D 타일 인터랙션 입력을 일관되게 바인딩.
Json & JsonUtilities	로컬 세이브게임 직렬화, 매치메이킹 페이로드 인코딩 및 외부 에이전트 파이프라인과의 메타데이터 교환.

2. 언리얼 엔진 5.7 전용 서버(Dedicated Server) 네트워크 권한 토폴로지

2.1. 단일 진실 공급원(Single Source of Truth) 원칙

WildCard의 멀티플레이어 아키텍처는 **권한 중심 전용 서버(Dedicated Server Authority)** 표준을 엄격하게 준수합니다. 클라이언트는 상태를 직접 변경할 수 없는 순수 뷰(Display/Input Layer)로 기능하며, 모든 게임 상태 변경(자원 소모, 카드 드로우, 타일 이동, 피해 판정, 턴 전이)은 반드시 서버의 HasAuthority() 검증을 통과한 후 서버 인스턴스에서만 최초 실행됩니다.

> **Dedicated Server Guardrail (보안 가드레일):**
> 클라이언트에서 송신되는 모든 의도(Intent)는 반드시 Server RPC와 WithValidation 핸들러를 거쳐야 합니다. 플레이어가 조작된 패킷으로 소유하지 않은 카드를 플레이하거나, 마나가 부족한 상태에서 스킬을 시전할 경우 서버 검증 단계에서 즉시 기각(Reject)됩니다.

2.2. RPC(Remote Procedure Call) 유효성 검증 체계

모든 플레이어 액션 제출은 AInGameController::Server_SubmitAction 또는 AInGamePlayerState::Server_SubmitMulligan을 통해 유입됩니다. 아래는 Dedicated Server 환경에서 클라이언트가 전달한 FWildActionData의 유효성을 3단계로 엄격하게 검증하는 C++ 구현체입니다.

Source/WildCard/InGame/Game/InGameController.cpp — C++ / UE 5.7

```cpp
bool AInGameController::Server_SubmitAction_Validate(FWildActionData ActionData)
{
    // 1단계: 패킷 기본 구조 검증 (위조된 ID 및 비정상 범위 차단)
    AInGamePlayerState* PS = GetPlayerState<AInGamePlayerState>();
    if (!PS || ActionData.PlayerID != PS->GetPlayerId())
    {
        UE_LOG(LogWildCard, Warning, TEXT("[Network Security] 클라이언트 위조 패킷 감지: 요청자 ID 불일치!"));
        return false;
    }

    // 2단계: 스킵(None) 액션은 항상 합법적 통과
    if (ActionData.ActionType == EWildActionType::None)
    {
        return true;
    }

    // 3단계: 카드 행동인 경우 플레이어의 실제 손패 소유 여부 사전 검증
    if (ActionData.CardInstanceID != -1 && !PS->IsCardInHand(ActionData.CardInstanceID))
    {
        UE_LOG(LogWildCard, Warning, TEXT("[Network Security] 유효하지 않은 카드 인스턴스 ID(%d) 제출 거부"), ActionData.CardI
        return false;
    }

    return true;
}

void AInGameController::Server_SubmitAction_Implementation(FWildActionData ActionData)
{
    // Authority 환경에서 PhaseComponent로 액션 검증 및 제출 위임
    if (AInGameMode* GM = GetWorld()->GetAuthGameMode<AInGameMode>())
    {
        if (GM->PhaseComponent)
        {
            GM->PhaseComponent->ProcessPrivateActionReady(this, ActionData);
        }
    }
}
```

2.3. 대역폭 최적화 및 프로퍼티 리플리케이션 전략

카드 게임의 특성상 상대방의 패(Hand) 정보는 클라이언트에 절대 복제되어서는 안 됩니다. 메모리 해킹 및 패킷 스니핑을 원천 차단하기 위해 PlayerCardComponent와 InGamePlayerState는 복제 조건을 정밀하게 분리하고 있습니다.

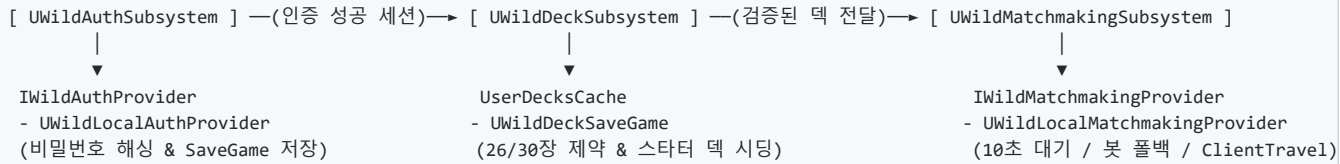
데이터 범주	변수명 및 컴포넌트	네트워크 복제 조건	설계 목적 및 보안 효과
개인 비밀 손패	CurrentHandDisplay (PlayerCardComp)	COND_OwnerOnly	본인 클라이언트에만 카드 목록 복제. 상대방 메모리에는 원천 미존재.
공개 손패 장수	HandCount (PlayerCardComp)	COND_None (전체 복제)	상대방 HUD에 뒷면 카드 개수만 표시하기 위한 메타 카운트.
공개 전장 상태	PublicMainActions (ChainComp)	COND_None (전체 복제)	체인 공방(Action_Public) 시점에 양 플레이어 모두에게 공개되는 행동.
시각/사운드 연출	Multicast_BroadcastVisualCue (InGameState)	NetMulticast (Unreliable)	게임플레이 로직과 무관한 코스메틱 큐. 네트워크 혼잡 시 프레임 드랍 방지.

OnRep 프레젠테이션 분리 원칙:
클라이언트에서 호출되는 OnRep_ 함수(예: OnRep_CurrentPhase, OnRep_Health)에는 게임 상태 변경 비즈니스 로직을 절대 포함하지 않습니다. 오직 위젯 텍스트 갱신, 애니메이션 트리거, 오디오 재생과 같은 로컬 프레젠테이션 계층 업데이트만 수행합니다.

3. GameInstance 전역 메타 서브시스템 아키텍처

WildCard는 매치 진입 전 계정 인증, 덱 편집, 매치메이킹 및 레벨 이동(Level Travel)을 처리하기 위해 언리얼 엔진의 **UGameInstanceSubsystem**을 활용하여 영속적인 메타 시스템 아키텍처를 구축하였습니다.

■ GameInstance 서브시스템 3계층 협력 다이어그램



3.1. UWildAuthSubsystem (계정 및 세션 관리)

UWildAuthSubsystem은 인터페이스 기반의 프로바이더 패턴(IWildAuthProvider)을 적용하여 설계되었습니다. 개발 및 로컬 테스트 단계에서는 UWildLocalAuthProvider가 로컬 UWildAccountSaveGame을 통해 계정을 관리하며, 향후 EOS(Epic Online Services)나 AWS Cognito/GameLift 백엔드로 전환할 때 서브시스템 내부 코드 수정 없이 프로바이더만 주입(Dependency Injection)할 수 있습니다.

Source/WildCard/Auth/WildAuthSubsystem.h — C++ / UE 5.7

```cpp
UCLASS()
class WILDCARD_API UWildAuthSubsystem : public UGameInstanceSubsystem
{
    GENERATED_BODY()

public:
    virtual void Initialize(FSubsystemCollectionBase& Collection) override;
    virtual void Deinitialize() override;

    UFUNCTION(BlueprintCallable, Category = "WildCard|Auth")
    void SignUp(const FString& Username, const FString& Password);

    UFUNCTION(BlueprintCallable, Category = "WildCard|Auth")
    void SignIn(const FString& Username, const FString& Password);

    UFUNCTION(BlueprintPure, Category = "WildCard|Auth")
    bool IsLoggedIn() const;

    UFUNCTION(BlueprintPure, Category = "WildCard|Auth")
    const FWildUserSession& GetCurrentSession() const;

    /** 외부에서 인증 프로바이더 교체 (추후 EOS / AWS 주입 시 사용) */
    void SetAuthProvider(TSharedPtr<IWildAuthProvider> InProvider);

    UPROPERTY(BlueprintAssignable, Category = "WildCard|Auth")
    FOnWildAuthResultSignature OnSignInCompleted;

private:
    TSharedPtr<IWildAuthProvider> ActiveProvider;
    FWildUserSession CurrentSession;
};
```

3.2. UWildDeckSubsystem (덱 빌딩 및 데이터 무결성 검증)

UWildDeckSubsystem은 유저별 다중 덱의 생성, 수정, 삭제(CRUD) 및 활성 덱 캐싱을 전담합니다. 매칭 진입 전 덱이 유효한지(정확히 26장 또는 30장인지, 해당 직업 카드로만 구성되었는지, 허용된 전술 스킬이 장착되었는지) ValidateDeck을 통해 검증합니다. 신규 계정일 경우 전사, 마법사, 도사 3종의 기본 스타터 덱을 자동 시딩합니다.

Source/WildCard/Deck/WildDeckSubsystem.cpp C++ / UE 5.7

```cpp
bool UWildDeckSubsystem::ValidateDeck(const FWildDeckData& InDeck, FText& OutErrorMessage) const
{
    // 1. 카드 매수 검증 (규격: 26장 또는 30장)
    if (InDeck.CardIDs.Num() != 26 && InDeck.CardIDs.Num() != 30)
    {
        OutErrorMessage = FText::FromString(FString::Printf(TEXT("덱은 정확히 26장 또는 30장이어야 합니다. (현재: %d장)"), InD
        return false;
    }

    // 2. 직업 전용 전술 스킬 장착 정합성 검증
    if (InDeck.SelectedTacticalSkillClass)
    {
        UTacticalSkillBase* CDO = InDeck.SelectedTacticalSkillClass->GetDefaultObject<UTacticalSkillBase>();
        if (CDO && !InDeck.CanEquipTacticalSkill(CDO->RequiredClass))
        {
            OutErrorMessage = FText::FromString(TEXT("해당 직업에서 장착할 수 없는 전술 스킬이 지정되었습니다."));
            return false;
        }
    }

    return true;
}
```

3.3. UWildMatchmakingSubsystem (스마트 대기열 & 클라이언트 트래블)

UWildMatchmakingSubsystem은 하스스톤 형태의 큐 대기 연출, 1초 타이머 틱, 타임아웃 봇 매칭 폴백 및 인게임 레벨 이동을 조율합니다. 로컬 프로바이더(UWildLocalMatchmakingProvider)는 10초 카운트다운 동안 타 유저 접속을 대기하며, 미접속 시 가상 봇 프로필을 구성하여 로컬 Dedicated Server 포트(7777)로 ClientTravel을 발동시킵니다.

매칭 상태 (EWildMatchmakingState)	발생 시점 및 트리거	시스템 동작 및 UI 상태
Idle	서브시스템 초기화 또는 매칭 취소 시	로비 대기 상태. 시작 버튼 활성화.
Searching	StartMatchmaking() 호출 시	1초 주기의 OnTimerTick 발송. 매칭 취소 가능.
MatchFound	상대 발견 또는 10초 타임아웃 봇 생성	매칭 다이얼로그에서 카운트다운 정지. 세션 IP 수신.
Connecting	ClientTravel(ServerAddress) 실행	인게임 맵 로딩 화면 전환 및 네트워크 접속 핸드셰이크.

4. 핵심 공통 데이터 계약 (BattleData.h 아키텍처)

Source/WildCard/InGame/Data/BattleData.h는 전 모듈에서 공유하는 불변 데이터 계약(Immutable Contracts)의 중심입니다. 네트워크 패킷 페이로드 최적화와 가비지 컬렉션 방지를 위해 경량 USTRUCT와 고성능 UENUM으로 구조화되어 있습니다.

4.1. 액션 전송 계약 (FWildActionData)

플레이어가 턴 중에 수행하는 모든 행위(유닛 소환, 주문 시전, 성유물 발동, 전술스킬, 타일 이동, 공격, 드로우)는 단일한 FWildActionData 구조체로 추상화되어 네트워크 파이프라인을 통과합니다.

Source/WildCard/InGame/Data/BattleData.h C++ / UE 5.7

```cpp
USTRUCT(BlueprintType)
struct WILDCARD_API FWildActionData
{
    GENERATED_BODY()

public:
    UPROPERTY(BlueprintReadWrite, Category = "Action")
    int32 PlayerID;                         // 행동을 발동한 플레이어 ID

    UPROPERTY(BlueprintReadWrite, Category = "Action")
    EWildActionCategory ActionCategory;     // Main (비공개 제출) vs Sub (공개 체인 대응)

    UPROPERTY(BlueprintReadWrite, Category = "Action")
    EWildActionType ActionType;             // SummonUnit, CastSpell, UnitMove, UnitAttack 등

    UPROPERTY(BlueprintReadWrite, Category = "Action|From Hand")
    int32 CardInstanceID;                   // 사용된 카드의 고유 인스턴스 ID

    UPROPERTY(BlueprintReadWrite, Category = "Action|From Hand")
    FName CardID;                           // 데이터 테이블 참조용 행 이름

    UPROPERTY(BlueprintReadWrite, Category = "Action|Target")
    int32 TargetPlayerID;                   // 대상 플레이어 ID

    UPROPERTY(BlueprintReadWrite, Category = "Action|Target")
    int32 TargetCardInstanceID;             // 대상 카드 인스턴스 ID

    UPROPERTY(BlueprintReadWrite, Category = "Action|On Board")
    TObjectPtr<ATile> SourceTile;           // 이동/공격 출발 타일

    UPROPERTY(BlueprintReadWrite, Category = "Action|On Board")
    TObjectPtr<ATile> TargetTile;           // 이동/소환/공격 대상 타일

    FWildActionData()
        : PlayerID(-1), ActionCategory(EWildActionCategory::None)
        , ActionType(EWildActionType::None), CardInstanceID(-1)
        , CardID(NAME_None), TargetPlayerID(-1), TargetCardInstanceID(-1)
        , SourceTile(nullptr), TargetTile(nullptr)
    {}
};
```

4.2. 비주얼 큐 계약 (FWildVisualCueData)

Dedicated Server에서 로직 판정이 완료된 후, 모든 클라이언트에 화려한 3D/2D 연출을 지시할 때 사용하는 코스메틱 전용 브로드캐스트 데이터 구조체입니다.

Source/WildCard/InGame/Data/BattleData.h C++ / UE 5.7

```cpp
USTRUCT(BlueprintType)
struct WILDCARD_API FWildVisualCueData
{
    GENERATED_BODY()

public:
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    FGameplayTag CueTag;                    // 연출 식별 태그 (예: Visual.TacticalSkill.Mage.SearchLibrary)

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    int32 InstigatorPlayerID = -1;          // 시전자 플레이어 ID

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    TWeakObjectPtr<AActor> SourceActor;     // 시전자 액터

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    TWeakObjectPtr<AActor> TargetActor;     // 피격 대상 액터

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    int32 SourceQ = -1;                     // 시전 축좌표 Q

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    int32 SourceR = -1;                     // 시전 축좌표 R

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    int32 TargetQ = -1;                     // 피격 축좌표 Q

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    int32 TargetR = -1;                     // 피격 축좌표 R

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    int32 NumericValue = 0;                 // 데미지, 힐량, 버프 수치

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    FName SourceID = NAME_None;             // 원인 카드/스킬 ID

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "VisualCue")
    FText CueMessage = FText::GetEmpty();   // 안내 텍스트

    FWildVisualCueData() = default;
};
```

5. 결론 및 다음 단계 요약

Volume 1에서는 WildCard 프로젝트의 전반적인 기술 스택, Dedicated Server 네트워크 보안 모델, GameInstance 서브시스템(인증, 덱, 매치메이킹) 및 전역 데이터 계약을 심층 분석하였습니다.

> **차기 볼륨 안내:**
> 이어지는 **Volume 2**에서는 인게임의 핵심 심장부인 AInGameMode, AInGameState, AInGamePlayerState의 상호작용과 6단계 턴/페이즈 상태머신(PhaseComponent), 그리고 LIFO 체인 스택 및 메인 행동 동시 처리 엔진(ChainComponent)을 분석합니다.